

Conceitos de Linguagens Formais

Módulo 8 – Ciência da Computação

Semana 1

Fillipe Resina

Professor de Programação

fillipe.resina@prof.inteli.edu.br

Objetivos da instrução

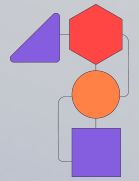
- Estudar outros tipos de autômatos e aprofundar os conhecimentos em Linguagens Formais e Autômatos
  - Pré-requisito para estudar compiladores

- Estudar linguagens mais elaboradas do que as Linguagens Regulares
  - **Linguagens Livres de Contexto**

- Aprofundar o nosso conhecimento em autômatos não-determinísticos

inteli

inteli



Organização do curso

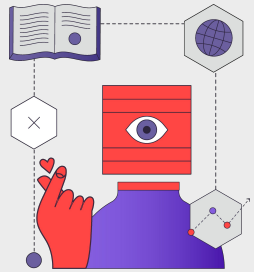
Organização das instruções de programação

- **Instruções sobre Linguagens Formais e Autômatos:**

  - **Conceitos de Linguagens Formais**
    - *Revisão sobre autômatos finitos. Estudar as linguagens livres de contexto, e construir autômatos de pilha (um AFN com memória);*

  - **Gramáticas**
    - *Formalismo importante para definição de linguagens livres de contexto e na construção das "regras" sintáticas de uma linguagem de programação;*

  - **Máquina de Turing**
    - *Compreender as limitações das linguagens livres de contexto. Reconhecimento de linguagens mais complexas e entender o que define um computador.*



inteli

Organização das instruções de programação



- **Instruções sobre construção de Compiladores:**
  - **Análise Léxica**
    - *Primeira etapa de um compilador. Reconhece se os símbolos pertencem à linguagem;*
  - **Análise Sintática**
    - *Segunda etapa de um compilador. Verifica se a sintaxe (ordem dos símbolos) está correta;*
  - **Análise Semântica**
    - *Verificações adicionais que não são possíveis na análise sintática (verificar se uma variável foi inicializada antes de ser usada, se os tipos dos operandos são compatíveis, etc);*
  - **Geração de Código**
    - *Gerar código executável a partir do código da linguagem que vamos construir :-)*

inteli

Com o Prof. Resina serão 3 ponderadas:

Sem 04: Autômatos, Linguagens Formais e Gramáticas

Sem 07: Análises Léxica e Sintática

Sem 08: Análise Semântica

Formato:

Desafio (pode envolver programação)

Presencial

Sem consulta (SEB)

50-60min

Mudança no horário do dia:

10h00 - 12h00: Dev

14h00 - 14h45/15h00: Desafio

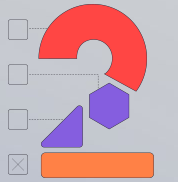
14h45 - 16h00: Autoestudo

16h00 - 16h15: Café

16h15 - 18h00: Instrução

PONDERADAS DO MÓDULO

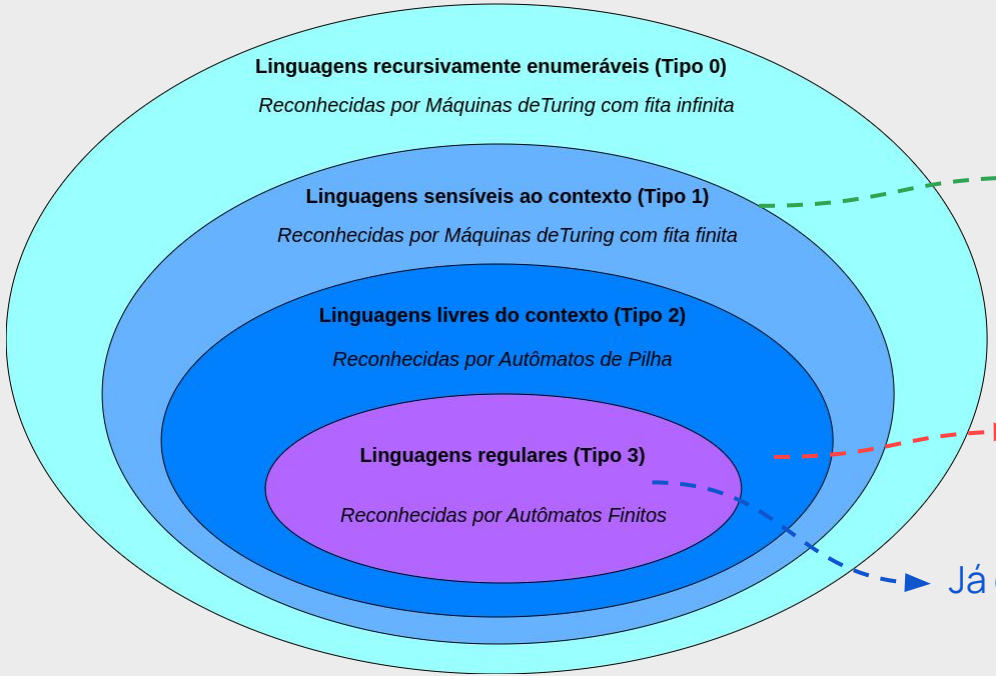
inteli



Hierarquia de Chomsky

Relembrando: hierarquia de Chomsky

- Classificação criada em 1959 pelo linguista Noam Chomsky:



Estudaremos muito em breve!



Assunto de hoje!
"Carinhosamente" conhecidas como LLCs



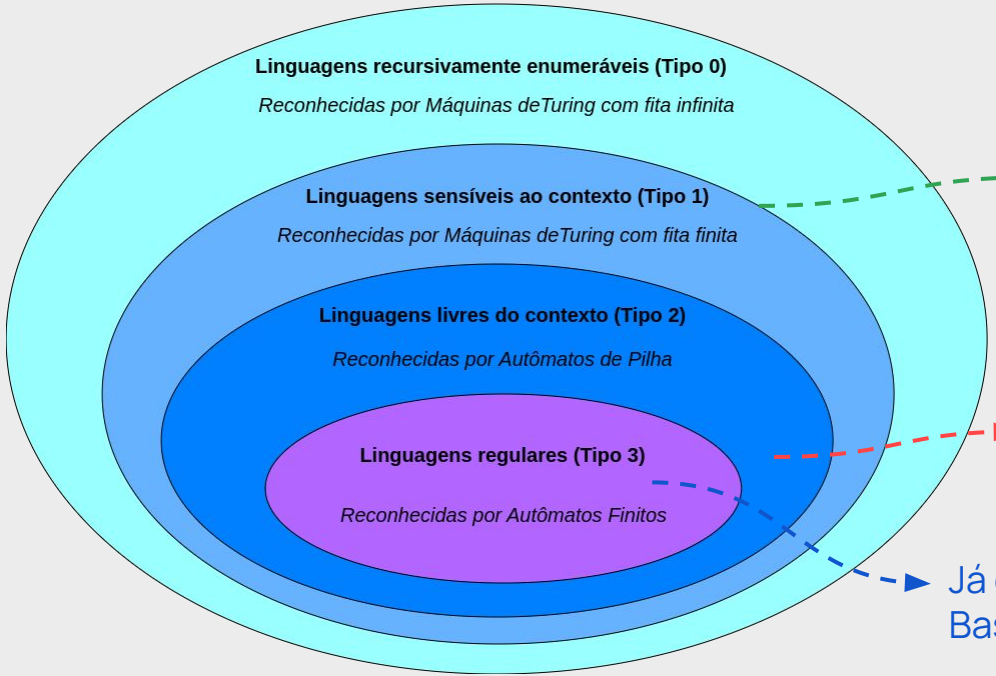
Já estudamos!



inteli

Relembrando: hierarquia de Chomsky

- Classificação criada em 1959 pelo linguista Noam Chomsky:





Estudaremos muito em breve!
Base da Análise Semântica (mas dá para fazer essa análise sem se aprofundar muito nesses tópicos)



Assunto de hoje!
Base da Análise Sintática!

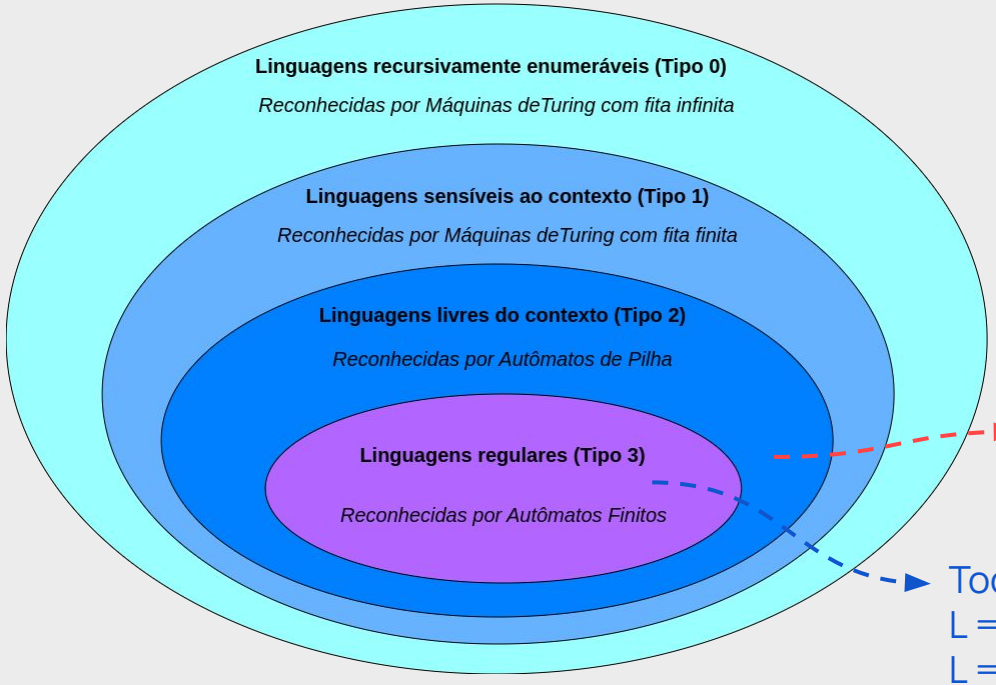


Já estudamos!
Base da Análise Léxica!

inteli

Relembrando: hierarquia de Chomsky

- Classificação criada em 1959 pelo linguista Noam Chomsky:



Alguns exemplos:

$L = \{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$
$L = \{ww^R \in \{a,b\}^*\}$
$L = \{a^n b^m c^k \in \{a,b,c\}^* \mid n, m \in \mathbb{N} \text{ e } k = n+m\}$
$L = \{a^n b^m c^k \in \{a,b,c\}^* \mid n, m, k \geq 0, n=m \text{ ou } n=k\}$

Todas as linguagens com nº finito de cadeias;
$L = \{a^n b^m \in \{a,b\}^* \mid n \geq 1 \text{ e } m \geq 1\}$
$L = \{w \in \{0,1\}^* \mid w \text{ termina em } 0\}$
$L = \{0^k \in \{0\}^* \mid k \text{ é múltiplo de 2 ou 3 }\}$

inteli

inteli



Revisão e conceitos iniciais

Revisão: alfabetos e cadeias

- Para simplificar a notação, utilizamos o conceito de alfabetos e cadeias para representar um "sinal de entrada":
  - **Alfabeto**: conjunto finito de elementos (símbolos), denotado geralmente por Σ;
    - Exemplos:

$$\Sigma_1 = \{a, b\}, \Sigma_2 = \{0, 1\}, \Sigma_3 = \{a, b, c, \ldots, x, y, z\}, \Sigma_4 = \{\#, =, +, -, *, /, 0, 1\}$$

  - **Cadeia** (string/palavra): é uma sequência finita de símbolos sobre um alfabeto Σ;
    - Formalmente: $x_1 x_2, \ldots, x_n, \text{ com } x_i \in \Sigma \text{ para } 1 \leq i \leq n$
    - Exemplos:
      - **1100** é uma cadeia sobre o alfabeto Σ = {0, 1}
      - **acaba** é uma cadeia sobre o alfabeto Σ = {a, b, c}
      - **inteli** é uma cadeia sobre o alfabeto Σ = {a, b, ..., z}

  - **Comprimento de uma cadeia**
    - Exemplos:

$$|1100| = 4$$

$$|acaba| = 5$$

$$|inteli| = 6$$

inteli

Revisão: alfabetos e cadeias

- Outras definições importantes:
  - **Concatenação**: $\alpha\beta = a_1 a_2 \ldots a_n b_1 b_2 \ldots b_m$
    - Exemplo: se α = 1100 e β = aabb, então: αβ = 1100aabb
  - **Potência de cadeia**: $w^k = \underbrace{ww \ldots w}_{k \text{ vezes}}$
    - Exemplos:
      - se α = 1100, então α^3 = 110011001100
      - se β = aabb, então β^2 = aabbaabb
  - **Potência de símbolo**: ajuda a simplificar a notação de cadeias com símbolos repetidos
    - Exemplos:
      - $a^5bc^3a = aaaaabccca$
      - $a^4b^4 = aaaabbbb$
  - **Cadeia vazia**: denotada pela letra grega épsilon **ε** (ε é um conceito, não um símbolo!)
    - É a cadeia de comprimento zero, isto é, **|ε| = 0**
    - Dada uma cadeia w sobre um alfabeto Σ, é sempre verdade que:

      wε = εw = w

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

$\Sigma^0 = \{\varepsilon\}$
$\Sigma^1 = \{0, 1\}$
$\Sigma^2 = \{00, 01, 10, 11\}$
$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

- **Fecho de Kleene** (também conhecido como operador "estrela"): é o conjunto de todas as cadeias sobre um alfabeto Σ, denotado por **Σ***:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

- **Fecho positivo:** denotado por **Σ⁺**, é definido como:

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ⁺ = {0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

inteli

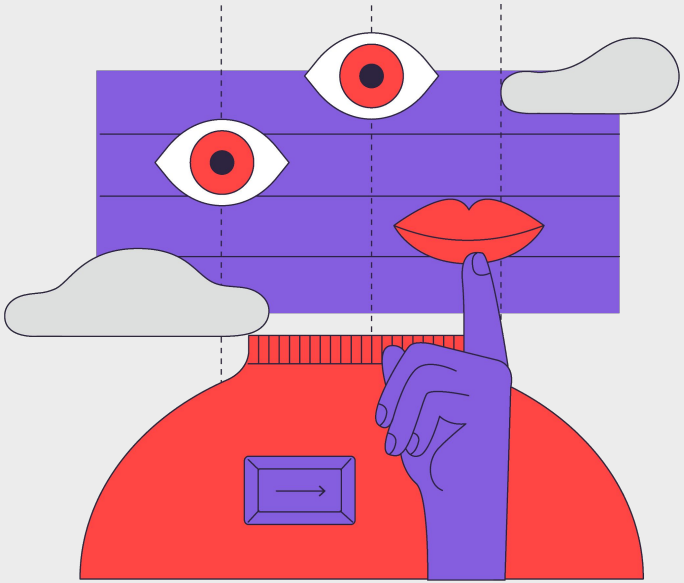
Revisão: linguagem

- Uma linguagem L sobre um alfabeto Σ é um subconjunto de Σ*: $L \subseteq \Sigma^*$
  - Ou seja: uma linguagem é um subconjunto de cadeias definidas a partir de um alfabeto!

  - Exemplos:
    - $L_1 = \{00, 01, 10, 11\}$
    - $L_2 = \{w \in \{0,1\}^* \mid w \text{ contém o mesmo número de 0s e 1s}\}$
    - $L_3 = \{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$
    - $L_4 = \{w \in \{0,1,2\}^* \mid \text{o quinto símbolo de } w \text{ é } 2\}$
    - $L_5 = \{1^n 01100^n \in \{0,1\}^* \mid n \geq 0\}$
    - $L_6 = \{w \mid w \text{ é um programa sintaticamente correto em Java}\}$
    - $L_7 = \{\varepsilon\}$
    - $L_8 = \{\ \}$

inteli

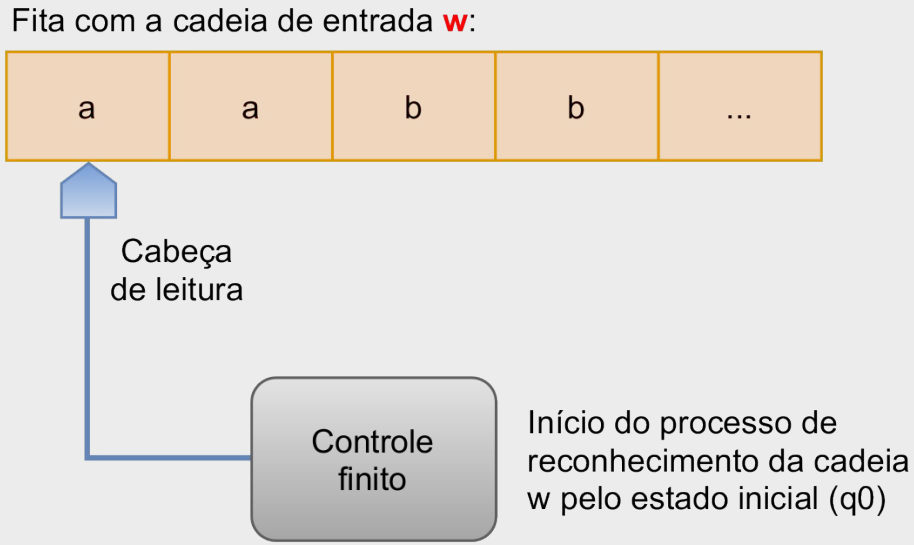
O que é um autômato?



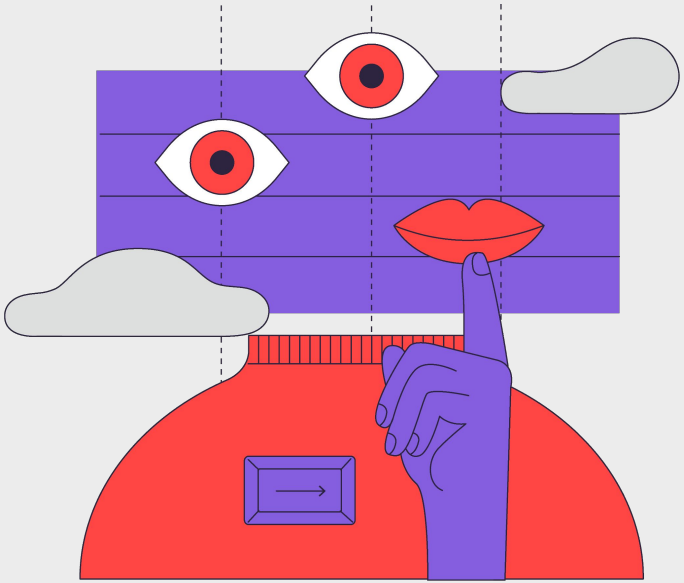
inteli

Representação de AFDs e AFNs (relembrando…)

- O “problema” é codificado como uma cadeia escrita em uma **fita** (de papel, por exemplo). Esta fita é **dividida em várias células**, e que cada **célula armazena somente um símbolo**:



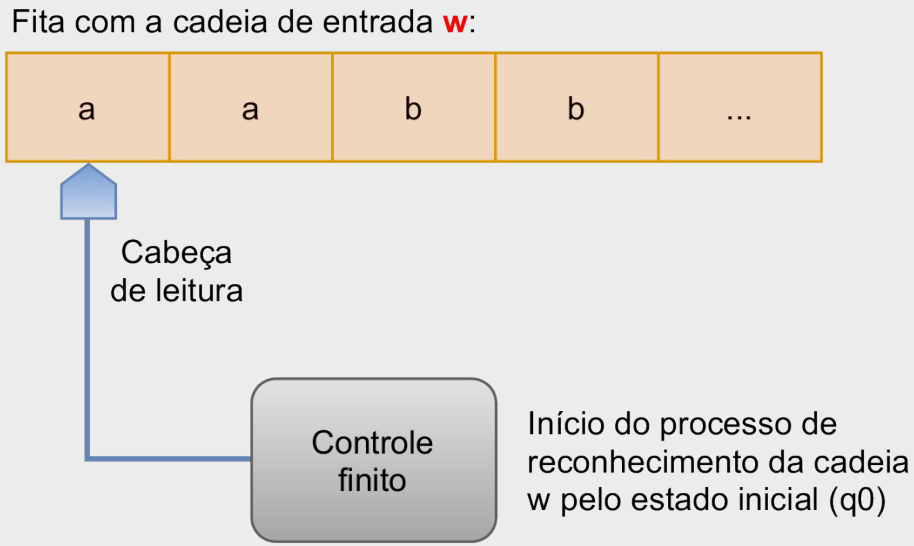
Como uma cadeia é aceita?



inteli

Autômato finito determinístico (AFD)

- Usaremos AFDs como **reconhecedores de linguagens**;
- Considere que o "problema" virá codificado como uma cadeia escrita em uma **fita** (de papel, por exemplo). Considere que essa fita é dividida em várias células, e que cada célula armazena somente um símbolo:



A cadeia será **aceita** pelo autômato quando:

- **Todos** os seus **símbolos** tiverem sido lidos;

  e ao mesmo tempo

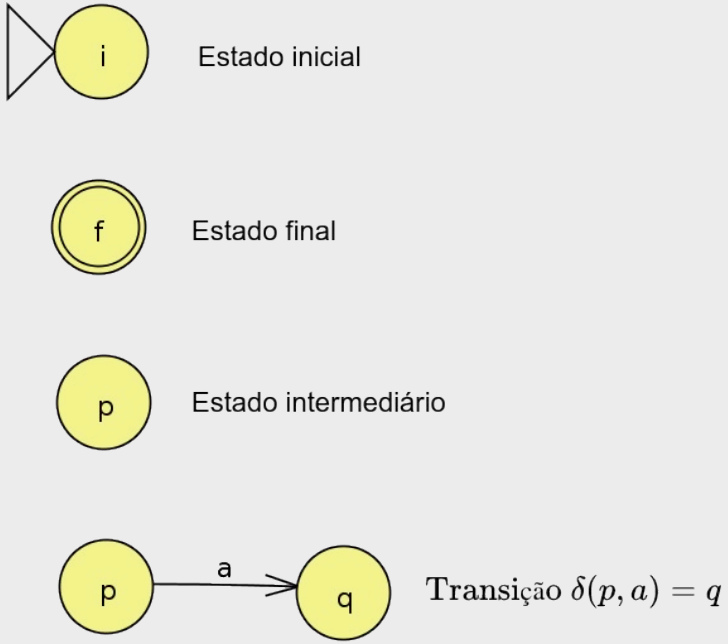
- o autômato **terminou** o processamento da cadeia em um **estado final**!

Caso contrário, dizemos que a cadeia **não foi aceita (rejeitada)**.

inteli

Autômato finito determinístico (AFD): notação gráfica

- Existe um (único) estado onde o processamento começa, chamado **estado inicial**;
- Podem existir um ou vários **estados finais** (ou **estados de aceitação**);
- Um estado que não é inicial e nem final é dito "**intermediário**".



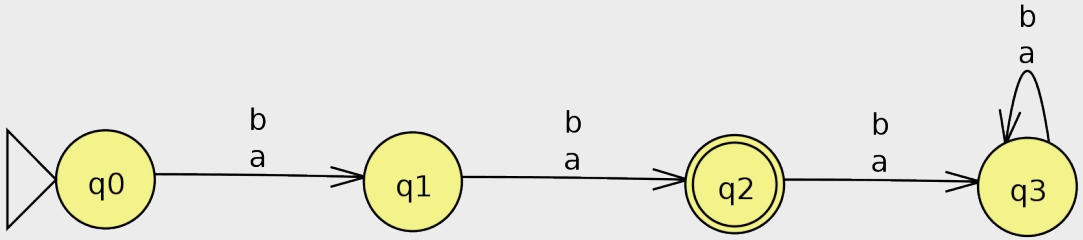
inteli

Linguagens regulares

- Dizemos que uma **linguagem é regular** quando ela é **reconhecida** por um **AFD**;
- De forma análoga, podemos afirmar que qualquer linguagem que um AFD reconhece é também uma linguagem regular;

    - Seja L_1 uma linguagem e M_1 um autômato: se M_1 reconhece L_1, denotamos: $\mathbf{L(M_1) = L_1}$.

- **Definição**: toda **linguagem finita** é **regular**!
    - Se uma linguagem possui um número finito de cadeias, é fácil imaginar que podemos construir um número finito de estados e transições para reconhecer todas as cadeias.
    - Exemplo: **L = {aa, ab, ba, bb}**



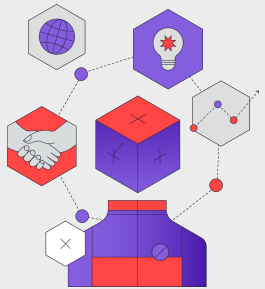
Note que o autômato é **determinístico**: para cada estado há uma (única) saída para cada símbolo do alfabeto.

inteli

Autômato finito não-determinístico (AFN): definição formal

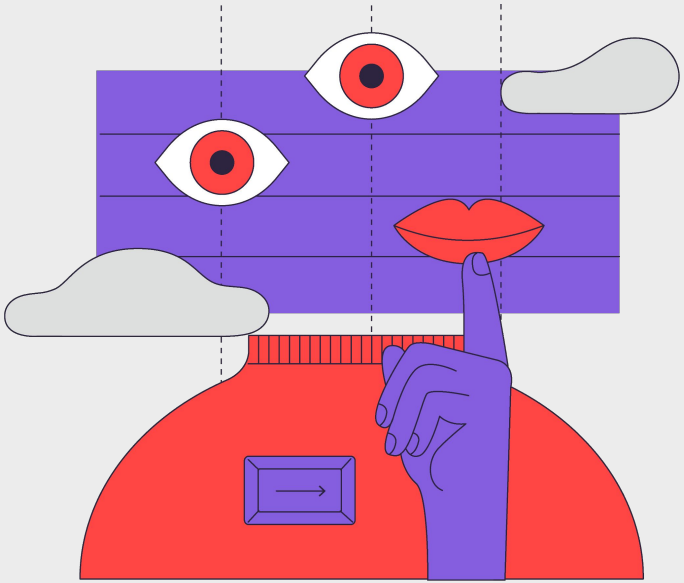
- Matematicamente, definimos um autômato finito **não-determinístico** M como uma 5-upla:
  M = (Q, Σ, δ, q0, F), onde:
  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto)
  - **δ** é a função de transição: $Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$
    onde $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$
  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

- Relembrando a definição de um AFD $\mathbf{M_2 = (Q, \Sigma, \delta, q0, F)}$:
  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto)
  - **δ** é a função de transição: $Q \times \Sigma \rightarrow Q$
  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$



inteli

Onde está a diferença?



inteli

Autômato finito não-determinístico (AFN): definição formal

- Matematicamente, definimos um autômato finito **não-determinístico** M como uma 5-upla:

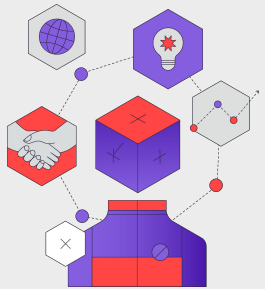
  M = (Q, Σ, δ, q0, F), onde:

  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto)
  - **δ** é a função de transição: $Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$

    onde $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$
  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

Nenhum (vazio) ou **vários** estados de destino para um mesmo símbolo lido!

Transições "vazias" (que **não leem** símbolos da entrada) são permitidas!



- Relembrando a definição de um AFD M_2 **= (Q, Σ, δ, q0, F)**:
  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto)
  - **δ** é a função de transição: $Q \times \Sigma \rightarrow Q$
  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

inteli

Definição importante: conjunto das partes

- O **conjunto das partes** (também conhecido como **conjunto potência**) de um conjunto A é o conjunto formado por todos os subconjuntos de A, normalmente denotado por $\mathcal{P}(A)$;

- **Exemplo**: Seja o conjunto A = {1, 2, 3}
  - O conjunto das partes de **A** é: $\mathcal{P}(A) = \{\varnothing, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$

OBS: note que todos os elementos de $\mathcal{P}(A)$ são conjuntos!

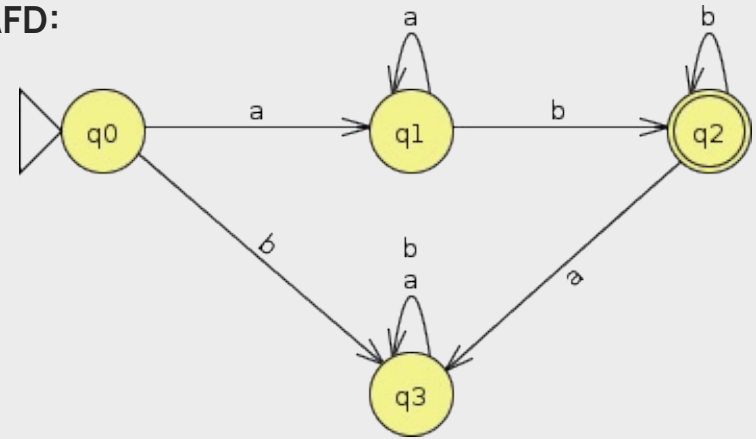
A cardinalidade do conjunto das partes sempre será igual a: $|\mathcal{P}(A)| = 2^{|A|}$



inteli

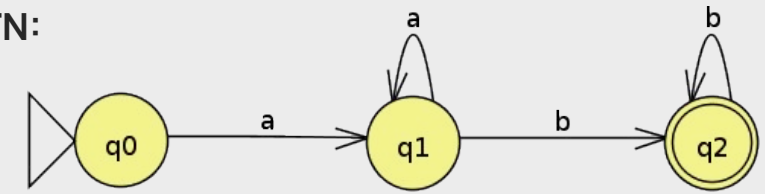
Exemplos de AFD e AFN para L = {$a^n b^m$ | n ≥ 1 e m ≥ 1}

AFD:



	a	b
q0	q1	q3
q1	q1	q2
q2	q3	q2
q3	q3	q3

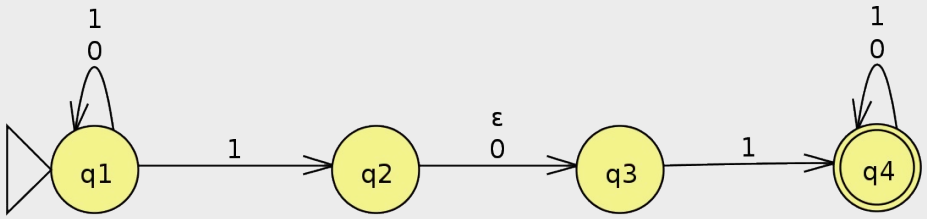
AFN:



	a	b	ε
q0	{q1}	∅	∅
q1	{q1}	{q2}	∅
q2	∅	{q2}	∅

inteli

Exemplo 2: L = {w ∈ {0,1}* | w possui 11 ou 101 como subcadeia}

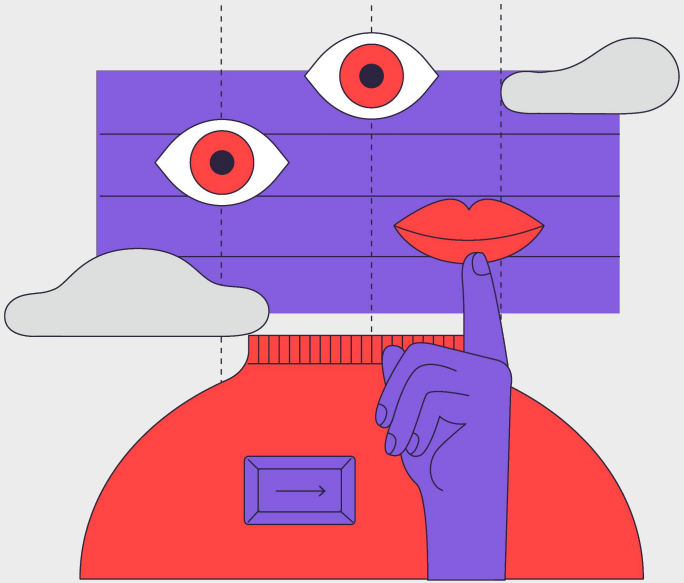


inteli



Autômato de Pilha

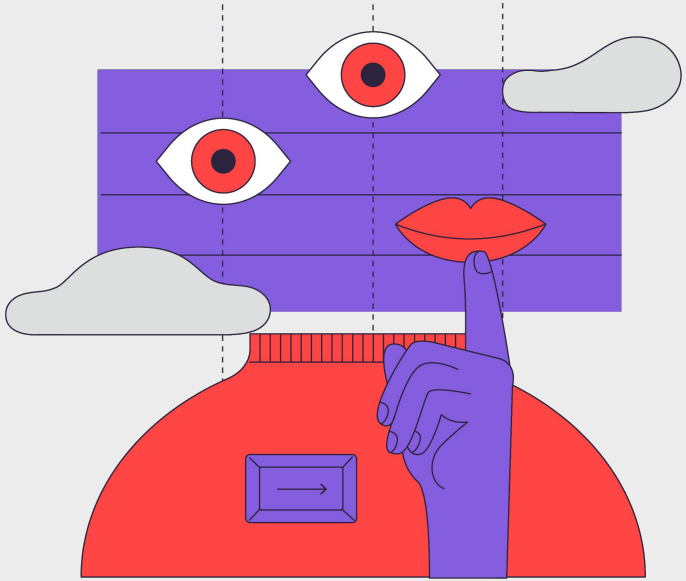
Qual a política de uma pilha?



Resp:LIFO!

inteli

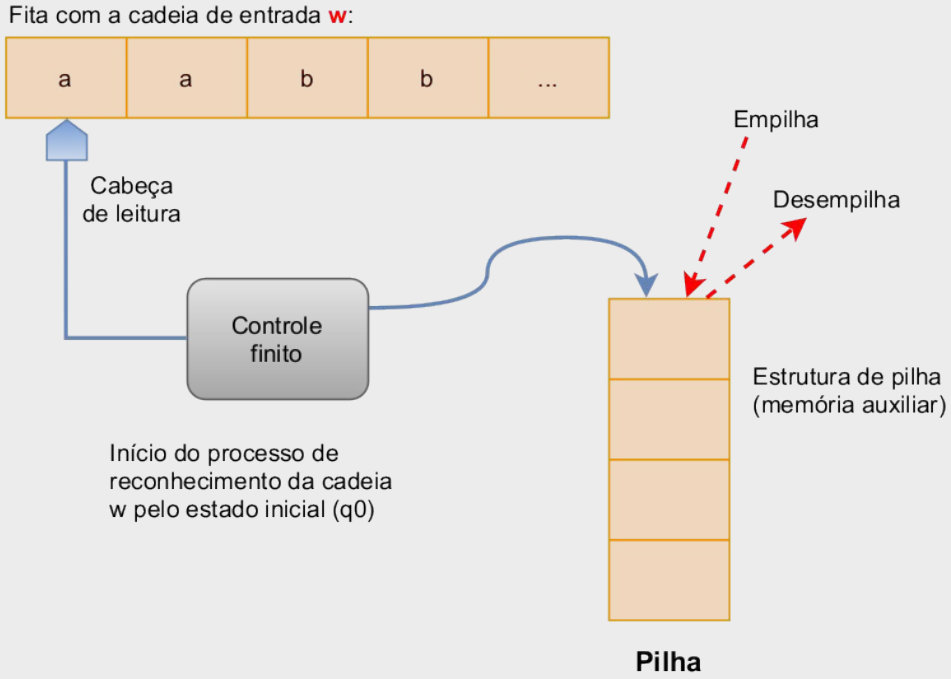
Quem se lembra de um exemplo de linguagem que não dá pra reconhecer com AFD?



inteli

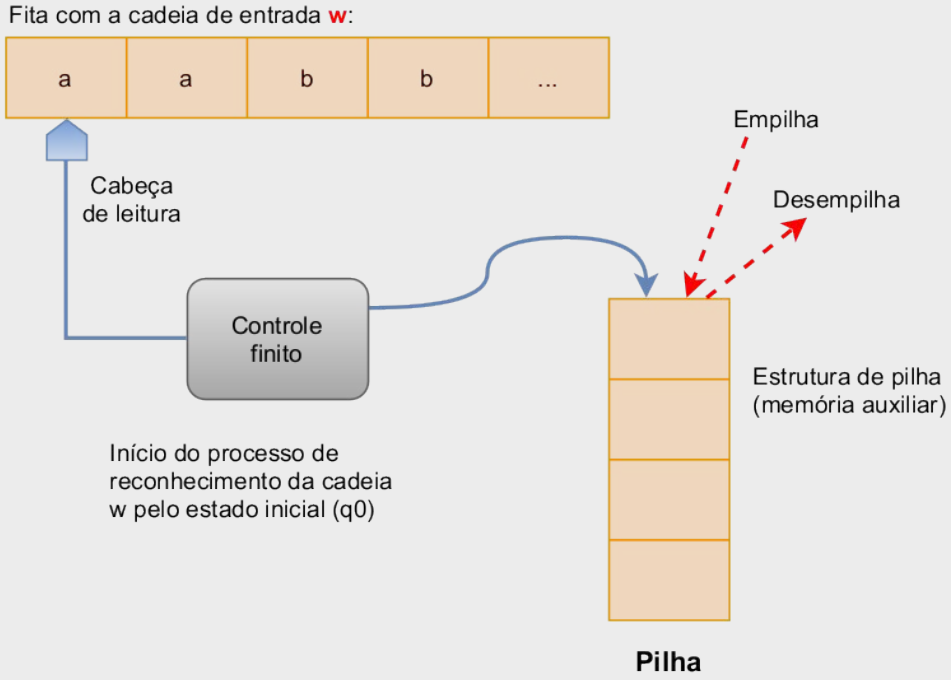
Autômato de pilha (AP)

- Um **AP** mantém a estrutura básica **de um AFN**, mas com **memória infinita**.
  - Ou seja, o AP é capaz de memorizar quantos símbolos forem necessários!
  - Facilita a modelagem do AP, pois não precisamos nos preocupar com limites de memória.



Autômato de pilha (AP)

- Um **AP** mantém a estrutura básica de **um AFN,** mas com **memória infinita**.
  - Ou seja, o AP é capaz de memorizar quantos símbolos forem necessários!
  - Facilita a modelagem do AP, pois não precisamos nos preocupar com limites de memória.



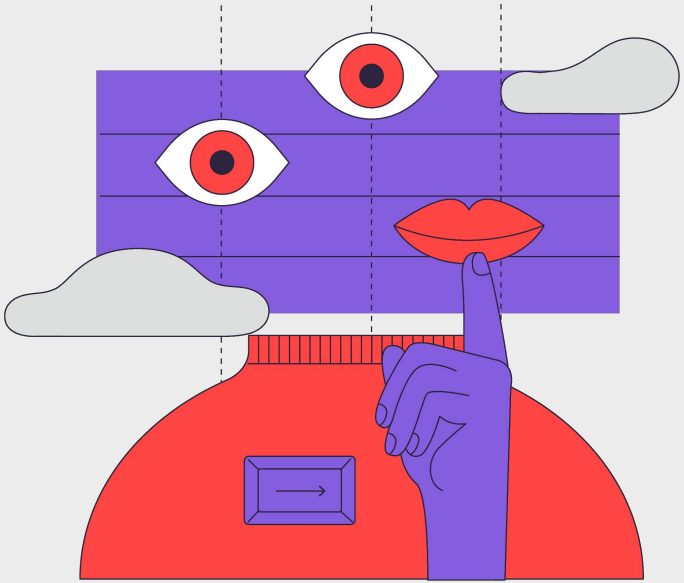
Você seria capaz de imaginar como podemos usar este modelo para reconhecer a seguinte linguagem?

$L = \{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$



inteli

O que é um autômato?



Resp: Sim! Quais?

inteli

Definição formal de um autômato de pilha (AP)

- Matematicamente, definimos um **autômato de pilha** M como uma 6-upla:

  M = (Q, Σ, Γ, δ, q0, F), onde:

  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto da linguagem / alfabeto da fita)
  - **Γ** é um conjunto de símbolos aceitos pela pilha (alfabeto da pilha)
  - **δ** é a função de transição: $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$

    em que $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ e $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$

  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

Entendendo a função de transição:

$$\delta : \underbrace{Q}_{\text{Estado anterior}} \times \underbrace{\Sigma_\epsilon}_{\text{Símbolo lido da entrada}} \times \underbrace{\Gamma_\epsilon}_{\text{Símbolo desempilhado}} \rightarrow \mathcal{P}(\underbrace{Q}_{\text{Próximo estado}} \times \underbrace{\Gamma_\epsilon}_{\text{Símbolo empilhado}})$$

inteli

Definição formal de um autômato de pilha (AP)

- Matematicamente, definimos um **autômato de pilha** M como uma 6-upla:

  M = (Q, Σ, Γ, δ, q0, F), onde:

  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto da linguagem / alfabeto da fita)
  - **Γ** é um conjunto de símbolos aceitos pela pilha (alfabeto da pilha)
  - **δ** é a função de transição: $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$

    onde $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ e $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$

  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

Entendendo a função de transição:

Significa que podemos ter transições que não lêem nada da fita!

Significa que podemos ter transições que não desempilham nada!

$$\delta : \underbrace{Q}_{\text{Estado anterior}} \times \underbrace{\Sigma_\epsilon}_{\text{Símbolo lido da entrada}} \times \underbrace{\Gamma_\epsilon}_{\text{Símbolo desempilhado}} \rightarrow \mathcal{P}(\underbrace{Q}_{\text{Próximo estado}} \times \underbrace{\Gamma_\epsilon}_{\text{Símbolo empilhado}})$$

Significa que podemos ter transições que não empilham nada!

inteli

Definição formal de um autômato de pilha (AP)

- Matematicamente, definimos um **autômato de pilha** M como uma 6-upla:
  M = (Q, Σ, Γ, δ, q0, F), onde:
  - **Q** é o conjunto finito de estados
  - **Σ** é um conjunto finito de símbolos (alfabeto da linguagem / alfabeto da fita)
  - **Γ** é um conjunto de símbolos aceitos pela pilha (alfabeto da pilha)
  - **δ** é a função de transição: $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$

    onde $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ e $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$

  - **q0** é o estado inicial, tal que $q0 \in Q$
  - **F** é o conjunto de estados finais, tal que $F \subseteq Q$

Note que: um **AP** é basicamente **um AFN com memória**. A função de transição reflete as operações possíveis com essa memória, no formato de uma pilha.

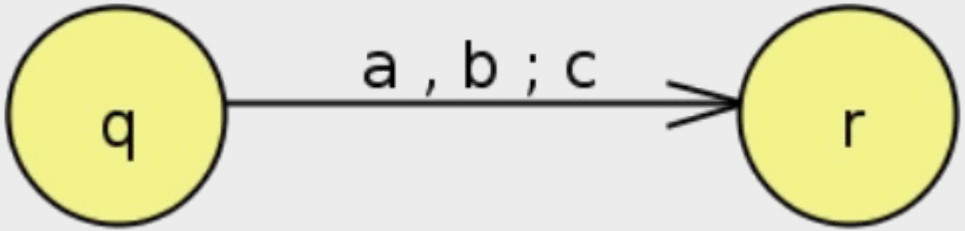
Por ser uma pilha, **não temos acesso "aleatório" a qualquer posição da memória**. Só será possível acessar o elemento do topo!

Curiosidade: existem APs **determinísticos**, mas eles **não são capazes de reconhecer todas as LLCs**. Por isso só vamos considerar a versão não-determinística, definida acima!

inteli

Representação das transições de um AP

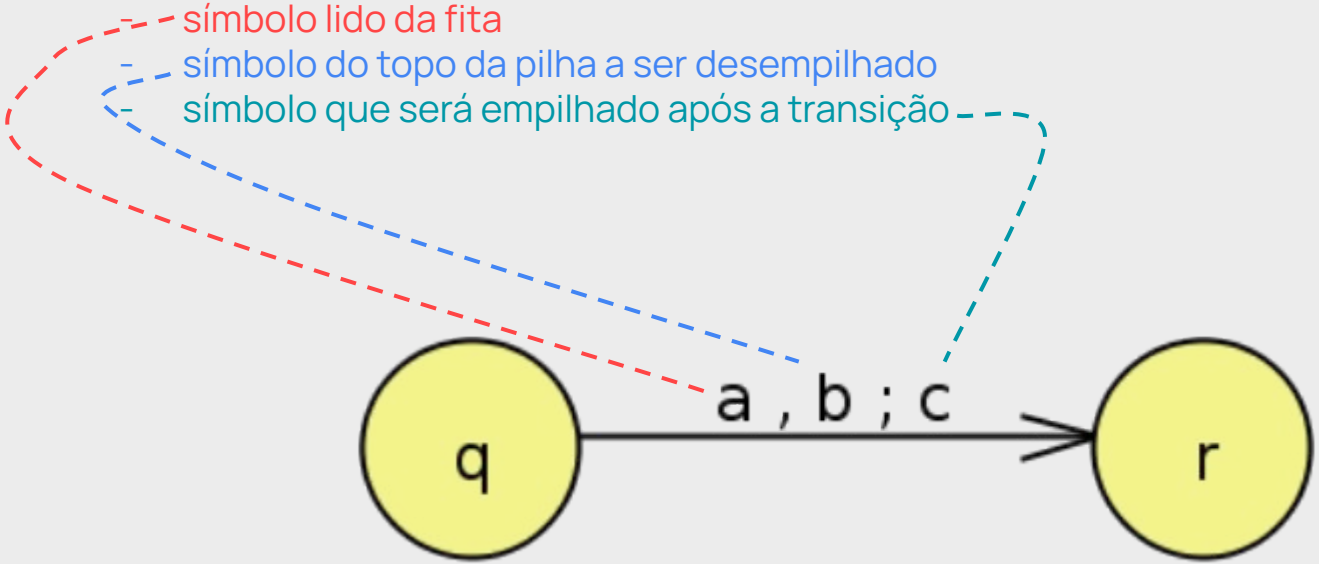
- Cada transição será representada por 3 valores, nesta ordem:
  - símbolo lido da fita
  - símbolo do topo da pilha a ser desempilhado
  - símbolo que será empilhado após a transição



inteli

Representação das transições de um AP

- Cada transição será representada por 3 valores, nesta ordem:
  - símbolo lido da fita
  - símbolo do topo da pilha a ser desempilhado
  - símbolo que será empilhado após a transição

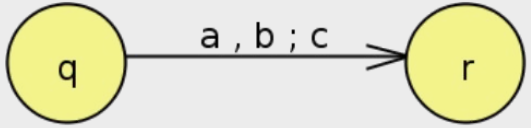


Em resumo, para que a transição aconteça, 2 condições **devem ocorrer simultaneamente**:

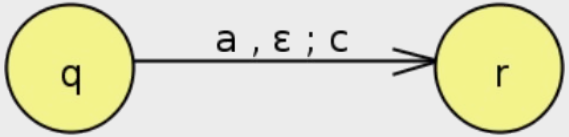
- o próximo símbolo da fita precisa ser **a**
- o topo da pilha deve conter **b**, e esse elemento será desempilhado

inteli

Interpretação das transições de um AP: alguns exemplos



Lê **a** da fita de entrada e desempilha **b** ao mesmo tempo. Em seguida, empilha **c**



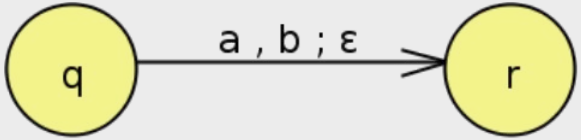
Lê **a** da fita de entrada sem desempilhar nada. Em seguida, empilha **c**



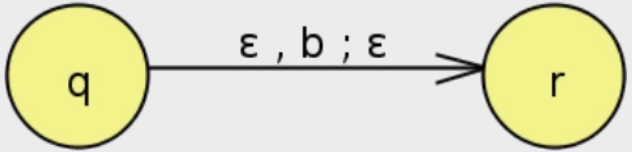
Não lê nada da fita de entrada, nem desempilha. Em seguida, empilha **c**

inteli

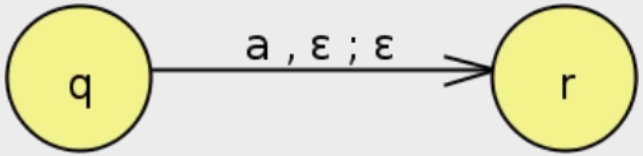
Interpretação das transições de um AP: alguns exemplos



Lê **a** da fita de entrada e desempilha **b** ao mesmo tempo. Não empilha nada



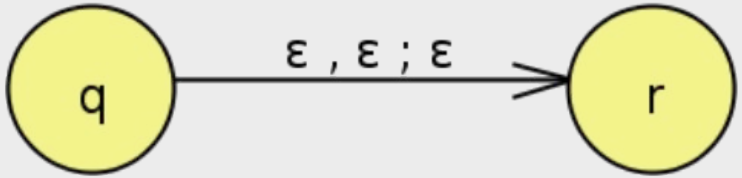
Não lê nada da fita de entrada e desempilha **b**. Não empilha nada



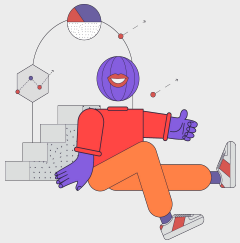
Lê **a** da fita de entrada, mas não desempilha nem empilha nada.
Essa transição é idêntica à transição de um AFN

inteli

Interpretação das transições de um AP: alguns exemplos



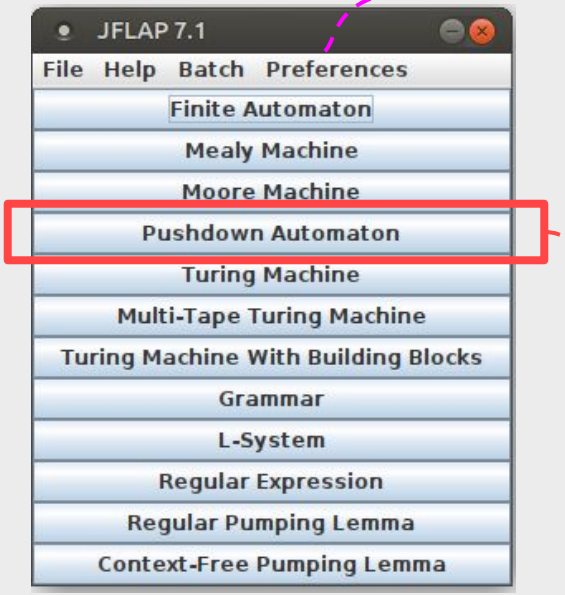
Não lê nada da fita de entrada, não desempilha nada e não empilha nada.
Transição similar a uma transição ε de um AFN!



inteli

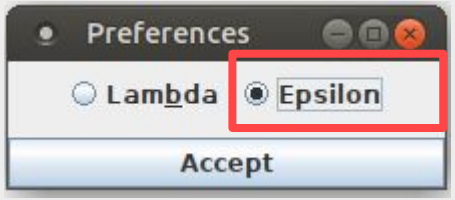
JFLAP

- **JFLAP:** software para construir e simular autômatos (finitos, de pilha, máquinas de Turing, etc).
  - Baixe a versão **JFLAP7.1.jar** em: https://www.jflap.org/jflaptmp/

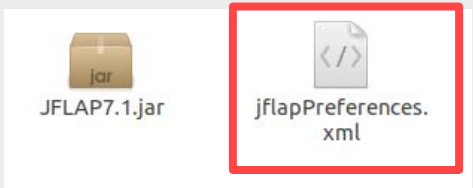


Autômato de Pilha

No menu **Preferences**, selecione a opção **Set the Empty String Character**, e mude-o para **Epsilon**, conforme a figura:



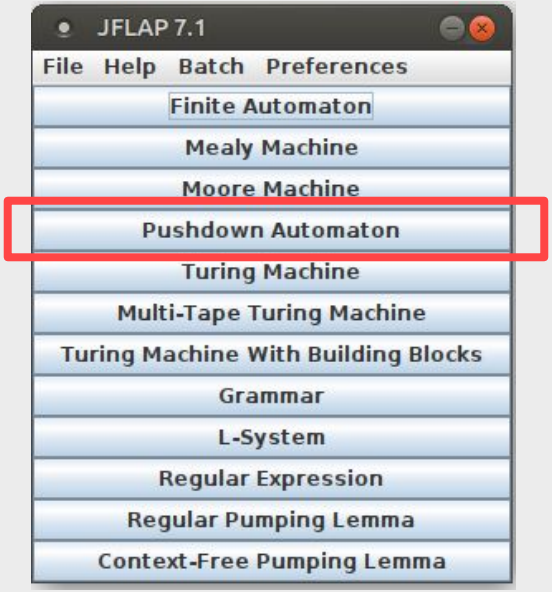
Note que será gerado um arquivo **xml**. Ele guarda as suas preferências do JFLAP, então é importante mantê-lo junto com o executável (arquivo **JAR**):



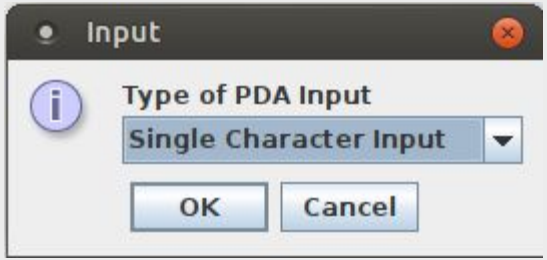
inteli

JFLAP

- **JFLAP:** software para construir e simular autômatos (finitos, de pilha, máquinas de Turing, etc).
  - Baixe a versão **JFLAP7.1.jar** em: https://www.jflap.org/jflaptmp/

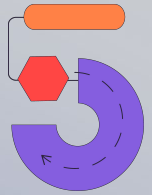


ATENÇÃO: ao clicar em **Pushdown Automaton**, o JFLAP vai perguntar qual tipo de entrada. Nós sempre vamos utilizar a segunda opção: **Single Character Input**



inteli

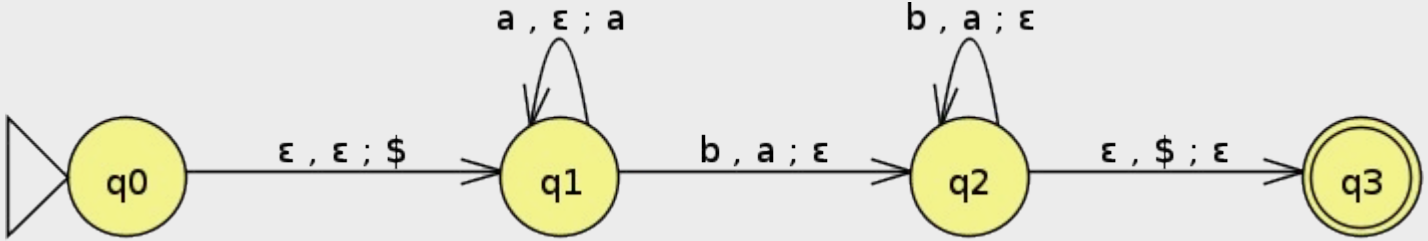
inteli



Exemplos de APs

AP que reconhece a linguagem: $\{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$

- Note que usamos o símbolo **$** para **marcar a base da pilha**. Ele nos indica quando a pilha está prestes a ficar vazia.
  - O JFLAP possui um marcador próprio (o símbolo **Z**), mas vamos seguir as boas práticas da literatura e ignorá-lo :-)



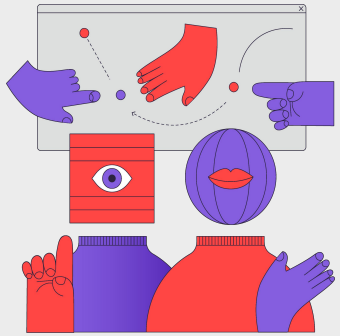
Curiosidade: a tabela de estados de transição de um AP tem o seguinte formato:

Entrada:	a			b			ϵ		
Pilha:	a	$	ϵ	a	$	ϵ	a	$	ϵ
q_0									$\{(q_1, \$)\}$
q_1			$\{(q_1, a)\}$	$\{(q_2, \epsilon)\}$					
q_2				$\{(q_2, \epsilon)\}$				$\{(q_3, \epsilon)\}$	
q_3									

inteli

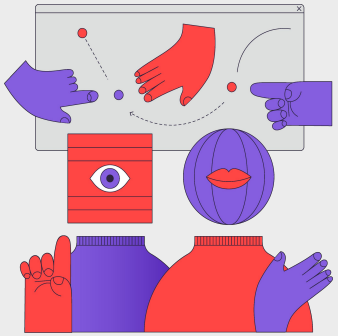
Construa e simule APs para reconhecer as seguintes LLCs

- $L_1 = \{0^n 1^{2n} \in \{a,b\}^* \mid n \geq 1\}$
- $L_2 = \{ww^R \mid w \in \{a,b\}^*\}$
- $L_3 = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i=j \text{ ou } i=k\}$
- $L_4 = \{a^n b^n c^n \mid n \geq 1\}$



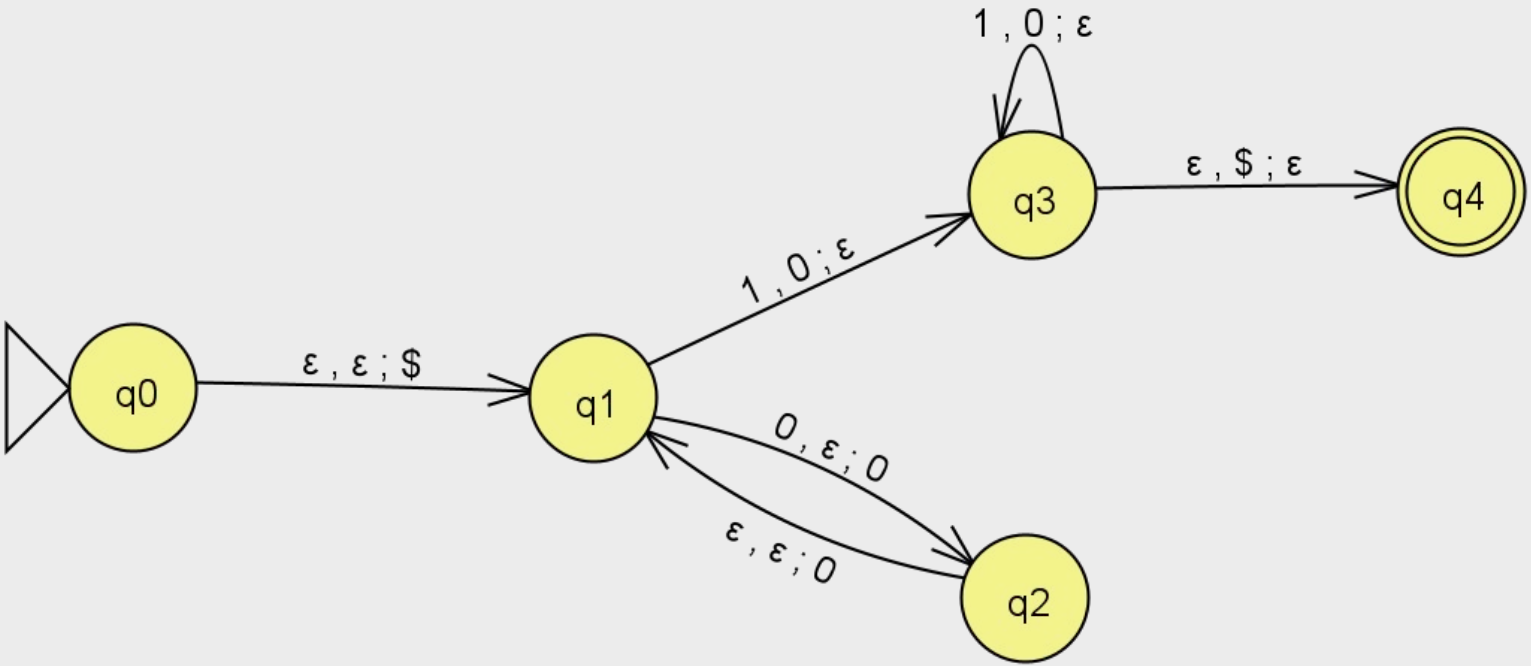
Construa e simule APs para reconhecer as seguintes LLCs

- $L_1 = \{0^n 1^{2n} \in \{a,b\}^* \mid n \geq 1\}$
  - Exemplos de cadeias: *011, 001111, 000111111*, etc.
- $L_2 = \{ww^R \mid w \in \{a,b\}^*\}$
  - Exemplos de cadeias: *aa, abba, baab, aabaabaa*, etc.
- $L_3 = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i=j \text{ ou } i=k\}$
  - Exemplos de cadeias: *aabbc, aabcc, abc, aabbcc*, etc.
- $L_4 = \{a^n b^n c^n \mid n \geq 1\}$
  - Exemplo de cadeias: *aabbcc, abc,* etc.



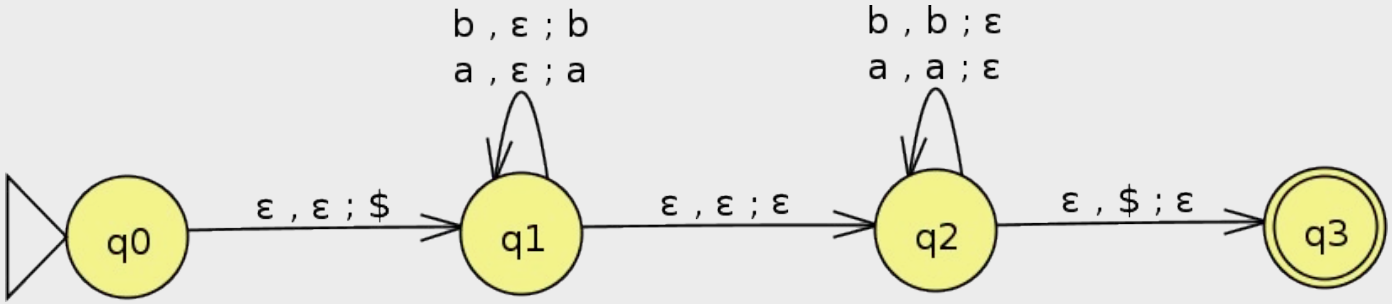
Construa e simule APs para reconhecer as seguintes LLCs

- $L_1 = \{0^n 1^{2n} \in \{a,b\}^* \mid n \geq 1\}$
  - Exemplos de cadeias: *011*, *001111*, *000111111*, etc.



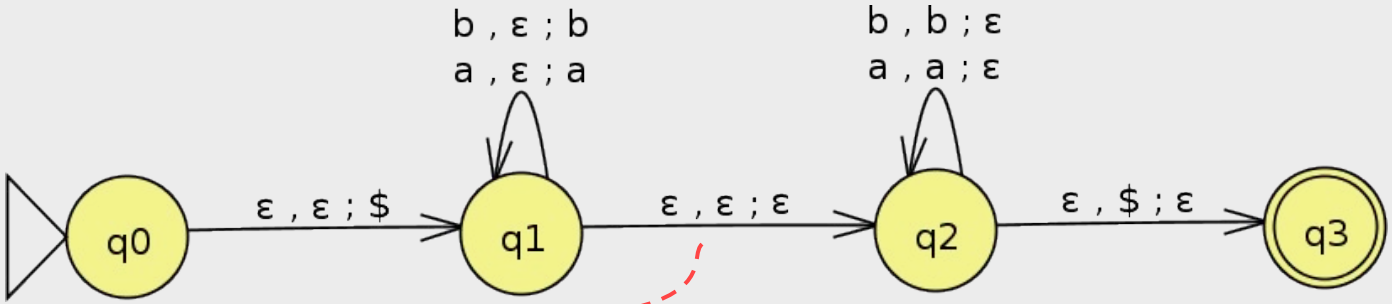
Construa e simule APs para reconhecer as seguintes LLCs

- $L_2 = \{ww^R \mid w \in \{a,b\}^*\}$
  - Exemplos de cadeias: *aa, abba, baab, aabaabaa*, etc.



Construa e simule APs para reconhecer as seguintes LLCs

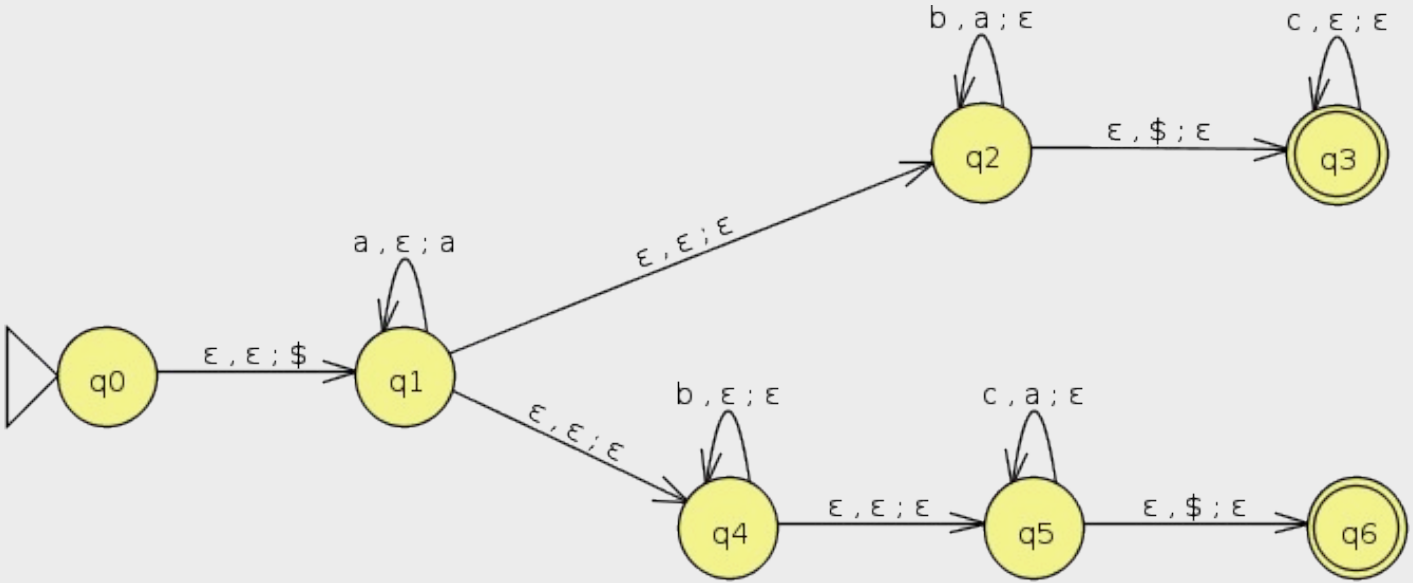
- $L_2 = \{ww^R \mid w \in \{a,b\}^*\}$
  - Exemplos de cadeias: *aa, abba, baab, aabaabaa*, etc.



Veja como o uso do não-determinismo foi útil para “chutar” onde fica o meio da cadeia!

Construa e simule APs para reconhecer as seguintes LLCs

- $L_3 = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i=j \text{ ou } i=k\}$
  - Exemplos de cadeias: *aabbc, aabcc, abc, aabbcc*, etc.



inteli



Considerações finais e conclusão

Linguagens não-livres de contexto

- Embora os **APs** sejam **mais poderosos computacionalmente** do que os **AFNs**, eles **não** são capazes de reconhecer qualquer tipo de linguagem!

- Alguns exemplos de **linguagens não-livres de contexto**:
  - $L = \{a^n b^n c^n \in \{a,b\}^* \mid n \geq 1\}$: seria necessário marcar um **a**, depois um **b** e depois um **c**, respectivo. Em seguida, seria necessário voltar ao primeiro **a** não marcado e recomeçar o processo.
    - As limitações no acesso à memória do AP não permitem operações deste tipo!

  - $L = \{ww \mid w \in \{a,b\}^*\}$: os símbolos inseridos na pilha ficam em ordem inversa. Seria necessário uma segunda pilha para deixar a primeira parte da cadeia na mesma ordem dos símbolos da segunda parte.
    - **Curiosidade**: no contexto de compiladores, esse problema é muito semelhante ao de verificar se uma variável utilizada em uma expressão foi inicializada anteriormente!



  - $L = \{a^n b^m c^k \in \{a,b,c\}^* \mid n, m \in \mathbb{N} \text{ e } k = nm\}$: seria necessário marcar um **a**, e depois ficar transitando entre os **b**s e **c**s, marcando os símbolos respectivos. Depois marcar o segundo **a**, e reiniciar todo o processo.

Spoiler alert: em breve estudaremos um outro tipo de autômato capaz de reconhecer essas linguagens!

inteli